Universidad del Valle de Guatemala

Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3092 Deep Learning, Semestre II 2026



Laboratorio 7

Roberto Barreda #23354

Guatemala, 5 de octubre de 2026

1. Investigación: embeddings en PyTorch y gensim

nn.Embedding(num_embeddings, embedding_dim, padding_idx, sparse) es una tabla que devuelve la fila de cada índice, lo mismo que multiplicar un one-hot por la matriz; padding_idx fija una fila en ceros sin gradiente y sparse=True genera gradientes con solo las filas del batch, que exigen SGD, SparseAdam o Adagrad. nn.Embedding.from_pretrained(embeddings, freeze) carga una matriz entrenada, fija si freeze=True. nn.EmbeddingBag busca y reduce una bolsa de índices con mode igual a mean, sum o max; con entrada 1D, offsets marca dónde empieza cada documento y no hace falta relleno. CBOW predice la palabra central desde el promedio del contexto y skip-gram predice cada vecino desde la central, lo que genera más pares y favorece a las palabras raras. Word2Vec usa W para la palabra como centro y W' como contexto o negativo, para no tener que dar un producto alto a una palabra consigo misma, y conserva W. Word2Vec es predictivo y recorre el corpus con SGD; GloVe ajusta el producto punto de cada par de vectores al logaritmo de su conteo de co-ocurrencia con mínimos cuadrados ponderados. Levy y Goldberg (2014) mostraron que SGNS factoriza una matriz PMI desplazada, así que ambos parten de las mismas estadísticas.

2. Decisiones del pipeline de preprocesamiento

WikiText-103 tiene 28,592 artículos, 1,809,468 líneas y 101.9 millones de tokens; el split de entrenamiento aporta 28,472 artículos y 101.4 millones de tokens. El tokenizador replica las decisiones de GloVe para que una palabra tenga la misma forma en los tres vocabularios: minúsculas, reconstrucción de los marcadores @-@, @,@ y @.@ ("role-playing", "3.5"), clíticos separados ("did n't", "it 's"), compuestos con guion y números como tokens, y puntuación descartada. Frente a NLTK en 10 oraciones, ambos coinciden en clíticos, guiones y números; difieren en la puntuación, en abreviaturas como "Mr." o "Jan.", en "gonna" y en correos electrónicos.

Con este tokenizador el split de entrenamiento queda en 85.3 millones de tokens y 717,893 tipos. La frecuencia contra el rango cumple la ley de Zipf entre los rangos 1 y 10^4 (pendiente -0.96) y cae más rápido en la cola (-1.69); las 10, 1,000 y 30,000 palabras más frecuentes cubren 24.5 %, 63.9 % y 94.6 % de los tokens. Los embeddings se entrenan con los primeros 16,822 artículos (50.0 millones de tokens). La Tabla 1 muestra el efecto de min_count; la base usa 5. Con sample=1e-4 el submuestreo descarta 96.2 % de "the", 94.1 % de "of" y 93.9 % de "and": aparecen junto a cualquier palabra, no informan sobre el significado y ocupan la ventana. Con ventana dinámica de 5 cada epoch tiene 287.8 millones de pares antes del submuestreo y 170.6 millones después.

Tabla 1. Tamaño del vocabulario y porcentaje de tokens desconocidos según min_count en el subconjunto de 50 M tokens.

min_count	1	2	5	10	20	50
Vocabulario	527,291	275,515	144,751	95,792	64,582	37,696
% tokens desconocidos	0.00	0.50	1.19	1.83	2.68	4.35

3. Iteraciones del modelo skip-gram

SGNS usa dos tablas nn.Embedding dispersas (W y W'), negativos muestreados de la distribución unigrama elevada a 0.75, pares generados en GPU, batches de 65,536 pares y SparseAdam con decaimiento lineal. La base usa d=100, ventana 5, 5 negativos, sample=1e-4, min_count=5, lr=0.02 y 5 epochs, y cada iteración cambia un hiperparámetro. Las analogías se evalúan con 3CosAdd dentro de un vocabulario compartido de 30,000 palabras presentes en todos los modelos, que cubre 12,181 de las 19,544 preguntas. La selección usa el promedio entre accuracy de analogías y Spearman en WordSim-353. La mejor iteración fue neg=15 (53.4 % y 0.673, contra 49.5 % y 0.674 de la base).

Tabla 2. Iteraciones de SGNS: tokens, vocabulario, pares por epoch (M), pérdida y métricas al final del epoch 5, tiempo y memoria pico de GPU.

Iteración	Tokens (M)	V	Pares (M)	Pérdida	Sem. %	Sin. %	Total %	WordSim	s/epoch	Total s	GPU GB
base	50.0	144,751	170.6	2.137	46.1	51.0	49.5	0.674	21.7	109	3.75
d=50	50.0	144,751	170.6	2.182	31.0	42.4	38.9	0.652	12.9	65	3.53
d=300	50.0	144,751	170.6	2.090	56.3	51.0	52.6	0.637	127.8	639	4.62

Iteración	Tokens (M)	V	Pares (M)	Pérdida	Sem. %	Sin. %	Total %	WordSim	s/epoch	Total s	GPU GB
corpus 25 %	12.5	67,472	41.9	2.119	32.6	31.3	31.7	0.647	4.9	24	1.62
corpus 50 %	25.0	99,285	84.7	2.130	39.9	43.3	42.2	0.656	10.2	51	2.20
ventana 2	50.0	144,751	86.9	1.992	41.0	55.4	51.0	0.663	11.2	56	3.12
ventana 10	50.0	144,751	303.7	2.210	46.4	49.6	48.6	0.679	39.3	197	5.19
neg=15	**50.0**	**144,751**	**170.6**	**3.046**	**51.5**	**54.2**	**53.4**	**0.673**	**45.3**	**227**	**3.94**
sample=1e-3	50.0	144,751	221.5	2.180	43.7	54.3	51.0	0.672	28.8	144	4.42
min_count=20	50.0	64,582	165.7	2.200	42.7	51.4	48.7	0.686	19.1	96	3.38
lr=0.01	50.0	144,751	170.6	2.131	45.2	53.3	50.8	0.681	22.2	111	3.74

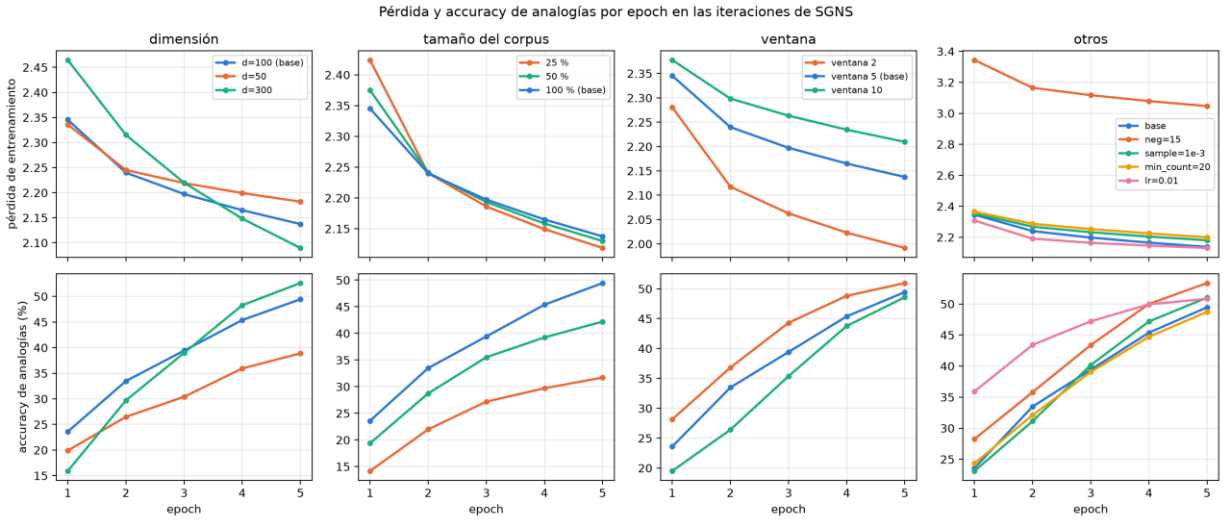


Figura 1. Pérdida de entrenamiento y accuracy de analogías por epoch, agrupadas por el hiperparámetro que cambia; la base aparece en azul.

¿Una pérdida más baja implica siempre mejores embeddings?

No. La ventana 2 terminó con la pérdida más baja (1.99) y 51.0 % en analogías, mientras que neg=15 terminó con la más alta (3.05) y el mejor resultado (53.4 %). El corpus de 25 % cerró con menos pérdida que la base (2.12 contra 2.14) y 17.8 puntos menos en analogías, porque con menos texto el modelo se ajusta mejor a esos pares sin aprender más relaciones. La pérdida depende de la tarea que define cada configuración (número de negativos, distancia del contexto), así que solo sirve para comparar epochs de una misma iteración; ahí sí baja mientras las analogías suben.

4. Verificación de la aritmética vectorial

analogia(a, b, c, k) normaliza los vectores, calcula b - a + c y devuelve las k palabras con mayor similitud coseno sin contar a, b ni c. Coincide con most_similar de gensim en palabras y similitudes para los tres modelos, y la evaluación vectorizada da el mismo accuracy que evaluate_word_analogies (65.49 % para GloVe con 30,000 palabras). La Tabla 3 resume 19 analogías propias de 6 tipos (género, país-capital, país-gentilicio, comparativos y superlativos, tiempos verbales y plurales), buscando dentro del vocabulario compartido. Los tres modelos resuelven los gentilicios y la mayoría de género y plurales; SGNS y gensim fallan en run : ran :: see : saw (rango 25 y 43) y en mouse : mice :: man : men (rango 16 y 192), donde GloVe acierta. Sin excluir las palabras de la consulta, king - man + woman devuelve king en los tres modelos.

Tabla 3. Analogías propias (19) y primera palabra sin excluir a, b y c en todo el benchmark.

Modelo	Top-1 %	Top-5 %	Rango mediano	Coseno con b-a+c	Sin excluir: c %	Sin excluir: b %	Sin excluir: d %
SGNS	73.7	84.2	1	0.730	85.6	4.3	9.2
gensim	68.4	78.9	1	0.722	84.9	5.0	9.0
GloVe	84.2	94.7	1	0.838	57.9	5.8	32.2

Tabla 4. Accuracy (%) por categoría de questions-words con 3CosAdd y 3CosMul en el vocabulario compartido, cobertura de preguntas y paralelismo (coseno medio entre vectores diferencia de la categoría).

Categoría	Cob. %	SGNS Add	SGNS Mul	gensim Add	gensim Mul	GloVe Add	GloVe Mul	Par. SGNS	Par. gensim	Par. GloVe
capital-common-countries	91	68.8	66.9	61.5	61.3	93.7	92.2	0.35	0.36	0.59
capital-world	26	62.3	57.9	53.5	50.1	86.3	83.8	0.28	0.29	0.56
currency	8	0.0	0.0	1.4	1.4	5.7	4.3	0.22	0.21	0.26
city-in-state	68	40.1	37.3	34.3	32.0	29.8	26.7	0.20	0.21	0.34
family	68	57.9	56.4	54.7	51.8	88.0	86.8	0.27	0.27	0.51
gram1-adjective-to-adverb	82	18.8	15.9	17.2	14.8	27.6	27.7	0.14	0.14	0.27
gram2-opposite	42	15.8	11.7	24.0	19.6	25.4	21.9	0.11	0.12	0.23
gram3-comparative	100	58.9	56.5	65.8	63.3	80.0	78.0	0.31	0.32	0.42
gram4-superlative	63	33.2	33.5	34.9	35.2	60.8	61.1	0.30	0.30	0.47
gram5-present-participle	82	43.0	38.7	43.1	39.3	72.1	71.5	0.18	0.17	0.32
gram6-nationality-adjective	81	87.1	84.8	87.4	85.5	97.4	95.8	0.49	0.50	0.75
gram7-past-tense	85	72.1	67.7	68.1	64.1	58.8	56.2	0.22	0.21	0.29
gram8-plural	79	58.9	53.3	52.5	44.3	77.9	73.8	0.18	0.18	0.31
gram9-plural-verbs	81	38.0	41.2	36.2	35.0	58.8	61.7	0.28	0.26	0.36
semánticas	42	51.5	48.5	44.9	42.6	60.3	57.8	0.26	0.27	0.45
sintácticas	79	54.2	51.5	54.1	50.9	67.6	66.2	0.24	0.24	0.38
total	62	53.4	50.6	51.3	48.3	65.4	63.6	0.25	0.25	0.40

Los vectores diferencia de una misma relación tienen coseno medio de 0.25 en SGNS, 0.25 en gensim y 0.40 en GloVe, contra 0.00 para pares al azar. El t-SNE de 514 palabras de 8 grupos (países, números, animales, verbos, profesiones, comida, cuerpo y tiempo) forma los grupos esperados con el mejor SGNS: 91 % de los 10 vecinos de cada palabra pertenecen a su grupo, y los ordinales y los días de la semana forman subgrupos.

5. Comparación entre los tres conjuntos de embeddings

Tabla 5. Comparación entre el mejor SGNS, Word2Vec de gensim y GloVe. Para GloVe, tiempo y hardware son los reportados por Pennington et al. (2014).

Métrica	SGNS	gensim	GloVe
Tokens del corpus	50,000,520	50,000,520	6,000,000,000
Vocabulario	144,751	144,751	400,000
Dimensión	100	100	100
Tiempo de entrenamiento	3.8 min (5 epochs)	4.8 min (5 epochs)	85 min para X y 14 min por iteración (d=300)
Hardware	RTX 4080 Laptop (GPU)	i9-13900HX, 12 hilos (CPU)	2 Xeon E5-2658, 32 núcleos
Analogías semánticas 3CosAdd (%)	51.5	44.9	60.3
Analogías sintácticas 3CosAdd (%)	54.2	54.1	67.6
Analogías total 3CosAdd (%)	53.4	51.3	65.4
Analogías semánticas 3CosMul (%)	48.5	42.6	57.8
Analogías sintácticas 3CosMul (%)	51.5	50.9	66.2
Analogías total 3CosMul (%)	50.6	48.3	63.6
Spearman WordSim-353	0.673	0.675	0.533
Spearman SimLex-999	0.28	0.314	0.298
Coseno medio entre vectores diferencia	0.25	0.253	0.404
% tokens OOV en AG News	2.87	2.87	1.08
F1 macro test AG News	0.9141	0.9133	0.9203

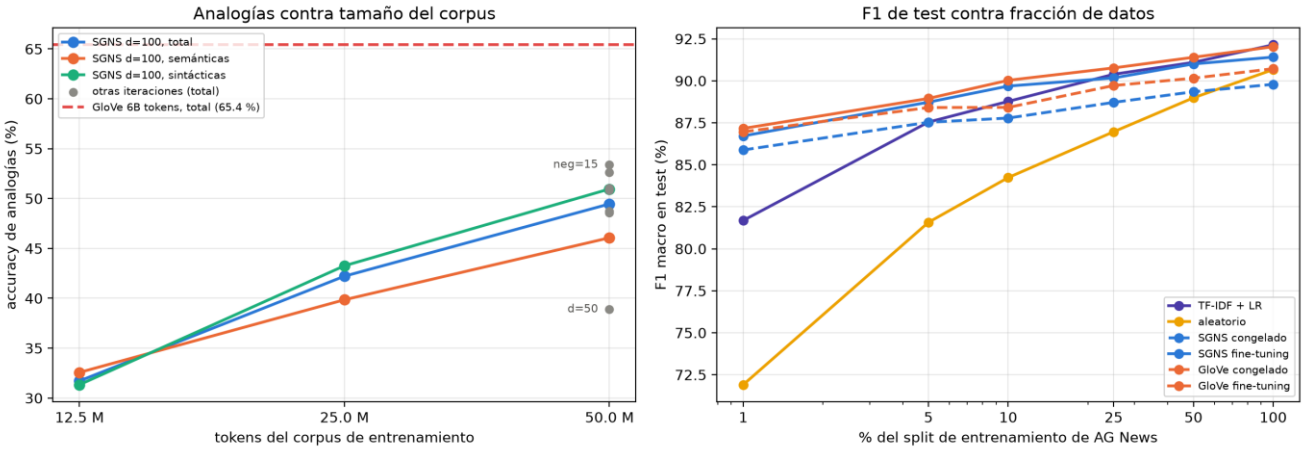


Figura 2. Izquierda: accuracy de analogías contra tokens del corpus para las iteraciones, con GloVe como referencia. Derecha: F1 macro de test en AG News contra fracción de datos de entrenamiento, por tipo de inicialización.

6. Resultados de clasificación en AG News

El pipeline aplica el mismo tokenizador a AG News, separa 10 % estratificado del entrenamiento para validación y deja el test oficial para el final. El clasificador es nn.EmbeddingBag (promedio, entrada 1D con offsets) seguido de un MLP con 256 unidades ocultas, entrenado con Adam y parada temprana sobre el F1 de validación. Los tokens fuera de vocabulario son 2.87 % para SGNS y gensim y 1.08 % para GloVe.

Tabla 6. Métricas macro (%) en validación, parámetros, tiempo de entrenamiento y F1 macro en test de la mejor variante de cada inicialización.

Modelo	Acc. val	Prec. val	Rec. val	F1 val	Parámetros	Entrenables	Tiempo s	F1 test
TF-IDF + LR	92.69	92.68	92.69	92.68	1,697,476	1,697,476	80	92.14
aleatorio	91.42	91.40	91.42	91.40	4,950,684	4,950,684	11	90.67
SGNS congelado	90.47	90.46	90.47	90.45	4,980,784	26,884	24	
SGNS fine-tuning	92.12	92.13	92.12	92.11	4,980,784	4,980,784	7	91.41
gensim congelado	89.92	89.90	89.92	89.90	4,980,784	26,884	23	
gensim fine-tuning	92.03	92.05	92.03	92.02	4,980,784	4,980,784	6	91.33
GloVe congelado	91.35	91.37	91.35	91.34	6,593,884	26,884	24	
GloVe fine-tuning	92.39	92.47	92.39	92.37	6,593,884	6,593,884	9	92.03

7. Discusión y análisis

¿Se cumple la aritmética vectorial en su modelo? ¿En qué categorías de analogías funciona mejor y en cuáles falla? ¿Hay diferencias entre analogías semánticas y sintácticas?

Se cumple en parte: el mejor SGNS acierta 53.4 % de las 12,181 preguntas con 3CosAdd y 14 de las 19 analogías propias. Funciona mejor en nacionalidad-adjetivo (87.1 %), tiempo pasado (72.1 %) y capitales comunes (68.8 %), y falla en monedas (0 %), opuestos (15.8 %) y adjetivo-adverbio (18.8 %). Las sintácticas (54.2 %) superan a las semánticas (51.5 %), porque estas dependen de entidades que aparecen poco en 50 millones de tokens, como monedas y capitales de países pequeños. 3CosMul quedó por debajo de 3CosAdd en los tres modelos.

¿Es el "aproximadamente igual" de king - man + woman = queen una igualdad?

No. Sin excluir la consulta, la palabra más cercana a b - a + c es c en 85.6 % de las preguntas (king para king - man + woman) y la respuesta correcta solo en 9.2 %. El coseno entre queen y el vector resultante es 0.74. Los vectores diferencia de una relación tienen coseno medio de 0.25 en SGNS y 0.40 en GloVe, contra 0 para pares al azar: comparten una dirección pero no son paralelos. La analogía acierta porque la respuesta es la más cercana después de excluir a, b y c, y las categorías con más paralelismo tienen más accuracy (Spearman de 0.51 en SGNS y 0.90 en GloVe).

¿Qué tan lejos quedó su modelo de GloVe? ¿Cuánto de la diferencia se explica por el tamaño del corpus y cuánto por el método o los hiperparámetros?
Quedó 12 puntos abajo en analogías (53.4 % contra 65.4 %) y arriba en WordSim-353 (0.673 contra 0.533). Duplicar el corpus de 12.5 a 25 millones de tokens sumó 10.5 puntos y de 25 a 50 millones 7.2, mientras que el mejor cambio de hiperparámetros sumó 3.9. GloVe usó 120 veces más tokens, unas 7 duplicaciones; con 2 puntos por duplicación se cubriría la brecha, así que la mayor parte se explica por el corpus. El método pesa en el tipo de relación: GloVe gana en capitales del mundo (86.3 % contra 62.3 %) y familia (88.0 % contra 57.9 %), y SGNS en ciudad-estado (40.1 % contra 29.8 %), tiempo pasado (72.1 % contra 58.8 %) y similitud.

¿Su implementación de SGNS se comporta parecido a la de gensim? Si hay diferencias, ¿a qué se deben?
Sí. Con el mismo corpus, vocabulario (144,751 palabras) e hiperparámetros, SGNS llega a 53.4 % en analogías y gensim a 51.3 %; quedan igual en sintácticas (54.2 % y 54.1 %) y en WordSim-353 (0.673 y 0.675), con tiempos de 3.8 min en GPU y 4.8 min en CPU. La diferencia está en las semánticas (51.5 % contra 44.9 %) y viene del optimizador: SparseAdam normaliza el paso de cada parámetro y una palabra rara recibe actualizaciones del mismo tamaño que una frecuente, mientras que el SGD de gensim la deja cerca de su inicialización. Por eso los vecinos de "king" en gensim son nombres raros ("harthacnut", "chlothar") y en SGNS son "queen", "throne" y "prince". gensim queda arriba en SimLex-999 (0.314 contra 0.280).

¿Qué efecto tuvieron la dimensión del embedding, el tamaño de ventana y el número de negativos? ¿Las ventanas pequeñas favorecen relaciones sintácticas o semánticas?
Con d=50 el total bajó de 49.5 % a 38.9 %; con d=300 subió a 52.6 %, pero WordSim bajó de 0.674 a 0.637 y cada epoch tardó casi 6 veces más. Con 15 negativos se obtuvo el mejor resultado (53.4 %) al doble de tiempo. La ventana 2 subió las sintácticas a 55.4 % y bajó las semánticas a 41.0 %; la ventana 10 dio 46.4 % y 49.6 %. Las ventanas pequeñas favorecen las relaciones sintácticas, porque los vecinos inmediatos reflejan la función gramatical; las grandes capturan el tema.

En AG News, ¿los embeddings preentrenados superaron a los aleatorios y al baseline TF-IDF? ¿Cuándo convino congelar y cuándo hacer fine-tuning? ¿Cómo cambia la respuesta con 1 % de los datos?
Con todos los datos, GloVe (92.0 % de F1 en test) y SGNS (91.4 %) con fine-tuning superaron a los aleatorios (90.7 %), pero no a TF-IDF (92.1 %), que usa bigramas y 424 mil rasgos. Fine-tuning superó a congelar en todas las fracciones, porque congelado solo se entrenan los 26,884 parámetros del MLP y los vectores no se adaptan a las noticias. Con 1 % de los datos (1,080 noticias) los preentrenados quedan entre 85.9 % y 87.2 %, contra 71.9 % de los aleatorios y 81.7 % de TF-IDF, y congelar queda a menos de un punto de fine-tuning con 185 veces menos parámetros entrenables. Congelar conviene con pocos datos y el fine-tuning con decenas de miles de ejemplos.

8. Conclusiones

1. El SGNS propio en PyTorch alcanzó 53.4 % en analogías, 2.1 puntos más que gensim con el mismo corpus y vocabulario.

2. El tamaño del corpus pesó más que los hiperparámetros: duplicar los tokens sumó entre 7.2 y 10.5 puntos y el mejor cambio de hiperparámetro sumó 3.9.

3. GloVe superó al mejor SGNS por 12 puntos en analogías, pero quedó por debajo en WordSim-353.

4. La aritmética vectorial es aproximada: sin excluir la consulta, la palabra c fue la más cercana en 85.6 % de las preguntas.

5. Con 1 % de AG News, los embeddings preentrenados superaron a los aleatorios por 14 a 15 puntos de F1.

Repositorio: https://github.com/bar23354/DL-Lab07